CLIMWEB

# Windows Setup, Backup and Restore Runbook

A practical guide to reproducing the ACMAD ClimWeb development environment, migrating PostgreSQL content and uploaded media, and resolving the startup issues encountered during setup.

Prepared 26 October 2026

# Purpose and scope

This runbook records the working procedure used to install ClimWeb on a Windows PC with Docker Desktop, correct Windows line-ending problems, initialize a fresh Wagtail site, export a smaller development backup from macOS, and restore that database and its uploaded files on Windows.

> **What is included**
>
> Docker-based development setup; environment configuration; CRLF repair; CSRF correction; fresh database bootstrapping; database and media backup; Windows restore; PostGIS topology conflict repair; and reliable service startup.

## Contents

1. Architecture and prerequisites
2. Clone and configure the Windows project
3. Repair Windows line endings
4. Initialize a fresh development database
5. Export database and media from macOS
6. Restore the backup on Windows
7. Resolve PostGIS restore conflicts
8. Start services without a database race
9. Verification and troubleshooting reference

> **Important distinction**
>
> The .psql.bin file stores database content such as pages, users, settings, snippets and submissions. The .tar file stores uploaded Wagtail images and documents. Source code is transferred through Git, not through these backups.

# 1. Architecture and prerequisites

ClimWeb runs as a group of Linux containers. Docker Desktop supplies the Linux environment, so the host Python installation is not used to run Django. Host Python is useful for generating secrets and repairing line endings.

| Service | Role |
|---|---|
| climweb_dev | Django development server and management commands |
| climweb_db_dev | PostgreSQL 15 with PostGIS |
| climweb_redis_dev | Cache and Celery message broker |
| climweb_celery_worker_dev | Background jobs |
| climweb_celery_beat_dev | Scheduled jobs |
| climweb_nginx | HTTP entry point on localhost ports 80 and 443 |

## Windows requirements

- Docker Desktop running Linux containers with the WSL 2 backend.
- Git for Windows.
- Python 3 available from PowerShell.
- Repository access for ACMAD-Niamey/climweb.

```
docker --version
docker compose version
git --version
python --version
```

# 2. Clone and configure the Windows project

### 1. Clone with Linux line endings

```
cd C:\
git -c core.autocrlf=false clone --branch ui-fixes --single-branch `
  https://github.com/ACMAD-Niamey/climweb.git
cd climweb
git config core.autocrlf false
```

Using core.autocrlf=false is essential because the Docker container executes repository shell and Python scripts as Linux files.

### 2. Create and edit the environment file

```
Copy-Item .env.sample .env
notepad .env
```

Use .env.sample, not .env.dev.sample. The latter points at host-local database services rather than the Compose service names.

```
CMS_DB_USER=climweb
CMS_DB_NAME=climweb
CMS_DB_PASSWORD=<strong-generated-value>
REDIS_PASSWORD=<strong-generated-value>
SECRET_KEY=<strong-generated-value>
CMS_DEBUG=True
CMS_BASE_URL=http://localhost
CMS_SITE_NAME=ACMAD ClimWeb
CMS_ADMIN_URL_PATH=cms-admin
TIME_ZONE=Africa/Niamey
CMS_DEFAULT_LANGUAGE_CODE=en
ALLOWED_HOSTS=localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=http://localhost,http://127.0.0.1
UID=1000
GID=1000
```

**CSRF syntax matters**

Every value in CSRF_TRUSTED_ORIGINS must include its scheme. The malformed value http:127.0.0.1 causes Django check 4_0.E001; the correct form is http://127.0.0.1.

# 3. Repair Windows line endings

The first startup failed with set: pipefail: invalid option name. This means carriage-return characters were added to Linux scripts. A second failure, /usr/bin/env: 'python\r': No such file or directory, showed that the executable Django file had the same problem.

## Repair shell scripts

```
python -c "from pathlib import Path; files=list(Path('.').rglob('*.sh'));
[p.write_bytes(p.read_bytes().replace(b'\r\n',b'\n').replace(b'\r',b'')) for p in files];
print('Fixed',len(files),'shell scripts')"
```

## Repair manage.py

```
python -c "from pathlib import Path; p=Path('climweb/src/climweb/manage.py');
p.write_bytes(p.read_bytes().replace(b'\r\n',b'\n').replace(b'\r',b'')); print('Remaining
CR:',p.read_bytes().count(b'\r'))"
```

The verification result must be Remaining CR: 0. Then rebuild the image:

```
docker compose --profile dev down --remove-orphans
docker compose --profile dev build --no-cache
docker compose --profile dev up -d
```

# 4. Initialize a fresh development database

On a completely empty database, project-specific seed commands expected a live HomePage and ProductIndexPage. The application therefore exited at CommandError: A live ProductIndexPage was not found. The minimum Wagtail tree was created using a one-off Django shell command.

**Why the first homepage slug failed**

Wagtail's starter content already used the slug home. The ClimWeb homepage was therefore created with the internal slug acmad-home and assigned as the default site's root page.

```
$bootstrap = @'
from wagtail.models import Page, Site
from climweb.pages.home.models import HomePage
from climweb.pages.products.models import ProductIndexPage
from climweb.base.models import ServiceCategory

root = Page.get_first_root_node()
home = HomePage.objects.first()
if home is None:
    home = HomePage(title="ACMAD Home", slug="acmad-home", hero_title="ACMAD")
    root.add_child(instance=home)
    home.save_revision().publish()

site = Site.objects.get(is_default_site=True)
site.root_page = home
site.hostname = "localhost"
site.port = 80
site.site_name = "ACMAD ClimWeb"
site.save()

products = ProductIndexPage.objects.first()
if products is None:
    products = ProductIndexPage(title="Products", slug="products",
                                banner_title="Explore our Products")
    home.add_child(instance=products)
    products.save_revision().publish()

ServiceCategory.objects.get_or_create(
    name="Regional Climate Center", defaults={"icon": "globe"}
)
print("Homepage:", home.id, home.slug)
print("Products:", products.id, products.slug)
'@
```

```
$encoded = [Convert]::ToBase64String(
  [Text.Encoding]::UTF8.GetBytes($bootstrap)
)
docker compose --profile dev run --rm climweb_dev manage shell -c `
  "import base64; exec(base64.b64decode('$encoded'))"
```

Base64 was used so PowerShell would pass the multiline Python program to Docker without altering quotes or newlines.

# 5. Export database and media from macOS

The smaller local macOS instance was selected as the migration source. From the repository directory in zsh:

```
docker compose --profile dev exec climweb_dev \
  python /climweb/web/src/climweb/manage.py dbbackup

docker compose --profile dev exec climweb_dev \
  python /climweb/web/src/climweb/manage.py mediabackup
```

The host receives both backup files in ./cms/backup/ because that directory is bind-mounted into the ClimWeb container.

```
ls -lht ./cms/backup
```

- Database file: default-<identifier>-<date>.psql.bin
- Media file: <identifier>-<date>.tar

> **Do not unpack the application backups**
>
> If the two files were transported inside an outer ZIP, extract only that ZIP. Leave the .psql.bin and .tar files intact for the ClimWeb restore commands.

# 6. Restore the backup on Windows

### 1. Place the files in the mounted backup directory

```
cd C:\Users\Acmad1\Desktop\climweb
New-Item -ItemType Directory -Force .\cms\backup
Copy-Item "$env:USERPROFILE\Downloads\*.psql.bin" ".\cms\backup\"
Copy-Item "$env:USERPROFILE\Downloads\*.tar" ".\cms\backup\"
```

### 2. Select the newest pair

```
$dbBackup = Get-ChildItem .\cms\backup\*.psql.bin |`
  Sort-Object LastWriteTime -Descending | Select-Object -First 1
$mediaBackup = Get-ChildItem .\cms\backup\*.tar |`
  Sort-Object LastWriteTime -Descending | Select-Object -First 1
$dbBackup.Name
$mediaBackup.Name
```

### 3. Stop services and preserve the local database

```
docker compose --profile dev down --remove-orphans
$stamp = Get-Date -Format "yyyyMMdd-HHmmss"
if (Test-Path ".\cms\db-data") {
  Rename-Item ".\cms\db-data" "db-data-before-restore-$stamp"
}
New-Item -ItemType Directory -Force ".\cms\db-data"
```

> **This step replaces local data**
>
> The old database directory is renamed rather than deleted, so rollback remains possible. Restore into the newly initialized database.

#### 4. Start only PostgreSQL and Redis

```
docker compose --profile dev up -d climweb_db_dev climweb_redis_dev
docker compose --profile dev exec climweb_db_dev `
  pg_isready -U climweb -d climweb
```

Continue only after the readiness check prints accepting connections.

# 7. Resolve the PostGIS topology conflict

The first database restore encountered cannot drop schema topology because other objects depend on it. The fresh PostGIS image had automatically installed optional topology extensions. The restore needs to recreate those objects from the backup.

```
docker compose --profile dev exec -T climweb_db_dev `
  psql -U climweb -d climweb -v ON_ERROR_STOP=1 `
  -c "DROP EXTENSION IF EXISTS postgis_topology CASCADE;" `
  -c "DROP EXTENSION IF EXISTS postgis_tiger_geocoder CASCADE;"
```

> **Keep the core extension**
>
> Do not remove the main postgis extension. Only postgis_topology and postgis_tiger_geocoder are removed before restoring.

## Restore database and media

```
docker compose --profile dev run --rm climweb_dev manage dbrestore `
  --input-filename "$($dbBackup.Name)" --noinput `
  --pg-options "--no-owner --no-acl"

docker compose --profile dev run --rm climweb_dev manage mediarestore `
  --input-filename "$($mediaBackup.Name)" --replace --noinput

docker compose --profile dev run --rm climweb_dev manage migrate --noinput
```

The migration command may report that models in background_task or products differ from migrations. Do not run makemigrations as part of environment restoration; that warning is not the startup blocker.

# 8. Start services without a database race

A full up --force-recreate restarted PostgreSQL and ClimWeb together. The TCP port opened before PostgreSQL completed recovery, so the application failed with FATAL: the database system is starting up. The reliable sequence is to wait for PostgreSQL, then start the application without restarting dependencies.

```
docker compose --profile dev exec climweb_db_dev `
  pg_isready -U climweb -d climweb

# Run only after the result says 'accepting connections'
docker compose --profile dev up -d --no-deps climweb_dev
docker compose --profile dev logs -f climweb_dev
```

When the log shows Running Development Server on 0.0.0.0:8000, press Ctrl+C. This closes only the log viewer.

```
docker compose --profile dev up -d --no-deps `
  climweb_celery_worker_dev climweb_celery_beat_dev
docker compose --profile dev restart climweb_nginx
docker compose --profile dev ps
```

# 9. Verification and troubleshooting reference

## Final checks

- All six services appear in docker compose --profile dev ps.
- The application log contains Running Development Server on 0.0.0.0:8000.
- The public site opens at http://localhost.
- The Wagtail dashboard opens at http://localhost/cms-admin/.
- Previously uploaded images and documents load from the restored media volume.

| Message | Meaning and action |
|---|---|
| set: pipefail: invalid option | CRLF in shell scripts. Convert carriage returns to LF and rebuild. |
| /usr/bin/env: python\r | CRLF in manage.py. Convert the file to LF. |
| CSRF 4_0.E001 | A trusted origin lacks http:// or https://. |
| No live ProductIndexPage | Fresh database lacks the minimum Wagtail page tree. Run the bootstrap. |
| cannot drop schema topology | Drop optional topology extensions, then retry dbrestore. |
| database system is starting up | Wait for pg_isready, then start climweb_dev with --no-deps. |
| pkg_resources is deprecated | Dependency warning only; it does not block startup. |
| CUIP document download ... 404 | Remote source documents are missing. The seed command skips them; unrelated to Docker health. |

## Useful commands

```
# Recent application logs
docker compose --profile dev logs --tail=150 climweb_dev

# Database readiness
docker compose --profile dev exec climweb_db_dev `
  pg_isready -U climweb -d climweb

# Stop services without deleting data
docker compose --profile dev down

# Start existing containers
docker compose --profile dev up -d
```

> **Separate data outside the media archive**
>
> The media backup covers Wagtail uploads. Data under cms/rcc-data and private export volumes are separate and must be copied independently when those datasets are required.

## Project references

The procedure is based on docker-compose.yml, .env.sample, climweb/docker/docker-entrypoint.sh, and docs/_docs/technical/backup-restore.md in the ACMAD ClimWeb repository.